TheFacebook: Topic 2 Data Structure Project Report

WIA1002 Data Structure Group Assignment

Group Members:Dong Hanyu(20251960202), Chen Linkai(20251960250), Wang Jingyu(20251960115), Li Zhongwu(20251960283), Hu Shinuo(20251960203)

1. Project Introduction

TheFacebook is a Java-based social networking application developed for Topic 2 of the WIA1002 Data Structure group assignment. The project simulates an early social media platform where users can create accounts, log in, edit profiles with uploaded avatars, search users by identity fields or hobbies, send and respond to friend requests, view mutual friends, receive recommendations, display first-, second-, and third-degree friends, create group conversations, chat with friends, navigate backward and forward through GUI history, and generate a simple analysis report.

The main purpose of the project is to apply data structures in a meaningful software system. A social network is suitable for this topic because user accounts need efficient searching, friendships need graph representation, friend requests need organized processing, group and chat records need dynamic collections, and session history can be represented using a LinkedList.

The application was first built as a console program and later extended with a simple Java Swing graphical user interface. Both interfaces use the same service layer and data files, so the GUI does not replace the original logic. Instead, it provides a more user-friendly way to demonstrate the same functions.

2. Project Objectives

- To design a simple social media system using object-oriented programming.
- To apply suitable data structures such as graph, queue, stack, ArrayList, LinkedList, and custom hash table.
- To store user, friendship, friend request, private chat, and group chat data permanently using CSV files.
- To implement the basic requirements of Topic 2, including registration, login, account editing, friend searching, mutual friends, friend requests, and mock data.
- To include extra features such as GUI, password hashing, data analysis, enhanced networking, friend chat, group chat, browsing history navigation, and degree-based friend display.

3. System Overview

The system is organized into packages so that each part of the program has a clear responsibility. The model package stores data objects, the service package controls the business logic, the datastructures package stores self-built or selected data structures, and the ui package handles user interaction.

Class	Main Responsibility

Main	Loads data and starts either the GUI or the console interface.
User	Stores user profile information, password hash, hobbies, career history, role, and avatar path.
Role	Defines USER and ADMIN roles.
FriendRequest	Represents one friend request from one user to another user.
Message	Stores one chat message with sender ID, timestamp, and text content.
FriendConversation	Stores one-to-one chat records between two friends.
GroupConversation	Stores group ID, group name, owner, members, and group messages.
SocialNetwork	Controls core functions such as login, registration, search, friend requests, recommendations, mutual friends, degree friends, chat, groups, reporting, and deletion.
CsvStorage	Handles reading and writing users, friendships, requests, private messages, groups, group messages, and reports.
PasswordUtil	Hashes passwords using SHA-256.
MyHashTable	Custom generic hash table used for fast lookup.
FriendGraph	Stores friendships and performs graph-related operations including BFS degree search.
BrowsingHistory	Original linked-node history used by the console traceback function.
LinkedListBrowsingHistory	GUI browsing history implemented with Java LinkedList, storing titles and content snapshots for History, Back, Forward, and clear.
ConsoleUI	Provides the original menu-based console interface.
TheFacebookGUI	Provides the Swing graphical interface, including profile avatar display/upload, hobby search, history restoration, and chat/group controls.

4. Basic Features and Implementation Methods

4.1 Data Storage

The program stores data in CSV files inside the data folder. users.csv stores account information, including avatar image paths. Uploaded avatar image files are copied into data/avatars so they stay inside the project folder. friendships.csv stores friendship edges, requests.csv stores pending friend requests, friend_messages.csv stores private chat records, groups.csv stores group information, and group_messages.csv stores group chat records. CsvStorage uses file reading and writing to make sure persistent data remains after the program closes.

4.2 Login, Registration, and Account Setup

The register method in SocialNetwork checks password confirmation, email or phone format, and duplicate username, email, or phone number. Passwords are hashed before storage. The login method searches by email first, then by phone number, and compares the hashed password.

4.3 Edit Account

Users can edit profile fields such as name, birthday, address, gender, relationship status, hobbies, and career history. Hobbies are stored in an ArrayList because the list can grow dynamically. Career history is stored in a Stack so the newest record can be shown first.

4.4 Add New Friends

A user can send a friend request to another user. Pending requests are stored in a Queue because requests can be reviewed in first-in-first-out order. When a request is accepted, FriendGraph adds an undirected friendship edge between the two users.

4.5 User Access Control

The Role enum separates USER and ADMIN accounts. Admin users can access the delete user function. This demonstrates basic access control using object-oriented programming.

4.6 Search for Friends

Users can search by name, username, ID, email, contact number, or hobby. The search uses linear search over all user records and case-insensitive substring matching. Each user's hobbies are checked one by one, so keywords such as Coding, Photography, Yoga, Travel, or Chess can find users with matching interests. Search results are placed in an ArrayList and sorted by name before being displayed.

4.7 View Account

The publicProfile method in User formats important profile information, including name, username, gender, age, address, relationship status, hobbies, career history, friend count, and role.

4.8 Mutual Friends

Mutual friends are found by getting the friend lists of two users from the graph and comparing them. If the same user ID appears in both lists, that user is a mutual friend.

4.9 Browsing History

LinkedListBrowsingHistory uses Java's LinkedList to store HistoryRecord objects for the current login session. Each record stores both a title and the actual content shown in the GUI text area. This allows Back and Forward to restore the previous or next displayed interface content instead of only printing a short message. The history list is cleared when the user logs out.

4.10 Friend Chat

FriendConversation represents one-to-one conversation between two friends. A private message can only be sent if both users already have a friendship edge in FriendGraph. Each message is stored as a Message object with sender ID, sending time, and text content. Private chat records are saved in friend_messages.csv.

4.11 Group Conversation

GroupConversation represents a group chat. It stores a group ID, group name, owner ID, member ID list, and group messages. Member IDs and messages are stored in ArrayLists because they need dynamic growth and simple ordered traversal. Groups are indexed in SocialNetwork by MyHashTable<String, GroupConversation> for fast lookup by group ID.

4.12 Degree Friends

FriendGraph uses breadth-first search to find first-degree, second-degree, and third-degree connections. First-degree users are direct friends, second-degree users are friends of friends, and third-degree users are one more level away. The GUI displays these three groups through the Degree Friends button.

4.13 Mock Data

The data files contain 30 normal users and 2 admin users. Additional CSV files also support friend messages, group records, and group messages. The mock users now include a wider range of hobbies such as Reading, Gaming, Art, Photography, Cooking, Swimming, Running, Dancing, Travel, Movies, Chess, Badminton, Football, Yoga, Writing, Volunteering, Hiking, and Debate. This allows hobby search to be tested immediately.

5. Data Structures Used

Data Structure	Used In	Reason for Selection
Custom Hash Table	usersById, usersByEmail, usersByPhone, usersByUsername, groupsById	Provides fast lookup and avoids direct use of java.util.HashMap.
Graph	FriendGraph	Friendship connections are naturally represented as users

		connected by edges. BFS supports recommendations and first-, second-, and third-degree display.
Queue	Friend requests	Friend requests can be handled in first-in-first-out order.
ArrayList	Search results, friends, hobbies, recommendations, group members, private messages, and group messages	Suitable for dynamic lists that are often traversed and displayed in order. Hobbies are also searched one by one during hobby search.
Stack	Career history	The newest career or study record can be pushed and shown first.
LinkedList	LinkedListBrowsingHistory	Browsing history is a sequence that supports moving backward, moving forward, removing old forward history, and restoring previous GUI text-area content.

6. Extra Features

Extra Feature	**Implementation Details**
Graphical User Interface	TheFacebookGUI.java uses Java Swing components such as JFrame, JPanel, JButton, JTextField, JPasswordField, JOptionPane, JScrollPane, and JTextArea.
Password Hashing	PasswordUtil uses SHA-256 so passwords are not stored in plaintext.
Data Analysis	SocialNetwork can generate analysis_report.txt with total accounts, admins, normal users, friendship count, pending requests, average age, and gender count.
Enhanced Networking	FriendGraph uses BFS to calculate connection degree up to the third degree and support friend recommendations.
Degree Friends	The Degree Friends GUI button displays first-degree, second-degree, and third-degree users

	separately.
Friend Chat	FriendConversation and Message store one-to-one messages between friends. Messages are saved in friend_messages.csv.
Group Conversation	GroupConversation stores group details, members, and messages. Group data is saved in groups.csv and group_messages.csv.
Browsing History Navigation	LinkedListBrowsingHistory stores current-session GUI actions and supports History, Back, Forward, and Logout clear.
Add Friend	Users are allowed to send friend requests to registered users, and accepted requests become graph edges.
Avatar Upload	The GUI lets users choose an image file. The image is copied into data/avatars and the saved path is written to users.csv.
Hobby Search	SocialNetwork.searchUsers performs linear search and checks each user's hobby ArrayList using case-insensitive substring matching.

7. Member Roles and Contributions

The following table divides the work among five members. The member names can be replaced with the actual names before submission. Each class is assigned to the member who is mainly responsible for writing and explaining it.

Member	**Main Role**	**Classes / Files Completed**	**Detailed Contribution**
Member 1 Name: Dong hanyu (20251960202)	Project leader and core service developer	SocialNetwork.java, ConsoleUI.java, FriendRequest.java, FriendConversation.java	Designed the program flow, connected storage and UI layers, implemented registration, login, update profile, search, friend request

			handling, mutual friends, recommendations, admin deletion, report generation, mock data loading, and private chat service methods.
Member 2 Name:Chen Linkai (20251960250)	Data model developer	User.java, Role.java, Message.java, GroupConversation.java	Created the user account object, role enum, message object, group conversation object, CSV conversion methods, age calculation, public profile display, hobbies ArrayList, and career Stack.
Member 3 Name:Wang Jingyu (20251960115)	Data structure developer	MyHashTable.java, FriendGraph.java, BrowsingHistory.java, LinkedListBrowsingHistory.java	Implemented the custom generic hash table, adjacency-list graph, BFS connection degree, mutual friend logic, recommendation sorting, linked-node traceback history, and LinkedList browsing history.
Member 4 Name:Li Zhongwu (20251960283)	Storage and security developer	CsvStorage.java, PasswordUtil.java, data/*.csv	Implemented CSV reading and writing for users, friendships, requests, private messages, groups, group messages, and reports; handled data folder management, SHA-256 password hashing, and mock data checking.
Member 5 Name:Hu Shinuo (20251960203)	User interface and testing developer	Main.java, TheFacebookGUI.java	Built the menu-based console interface, added the Swing GUI, connected buttons and

			dialogs to SocialNetwork methods, tested main user flows, and added GUI access to chat, group, history, and degree friend functions.

8. Problems Faced and Solutions

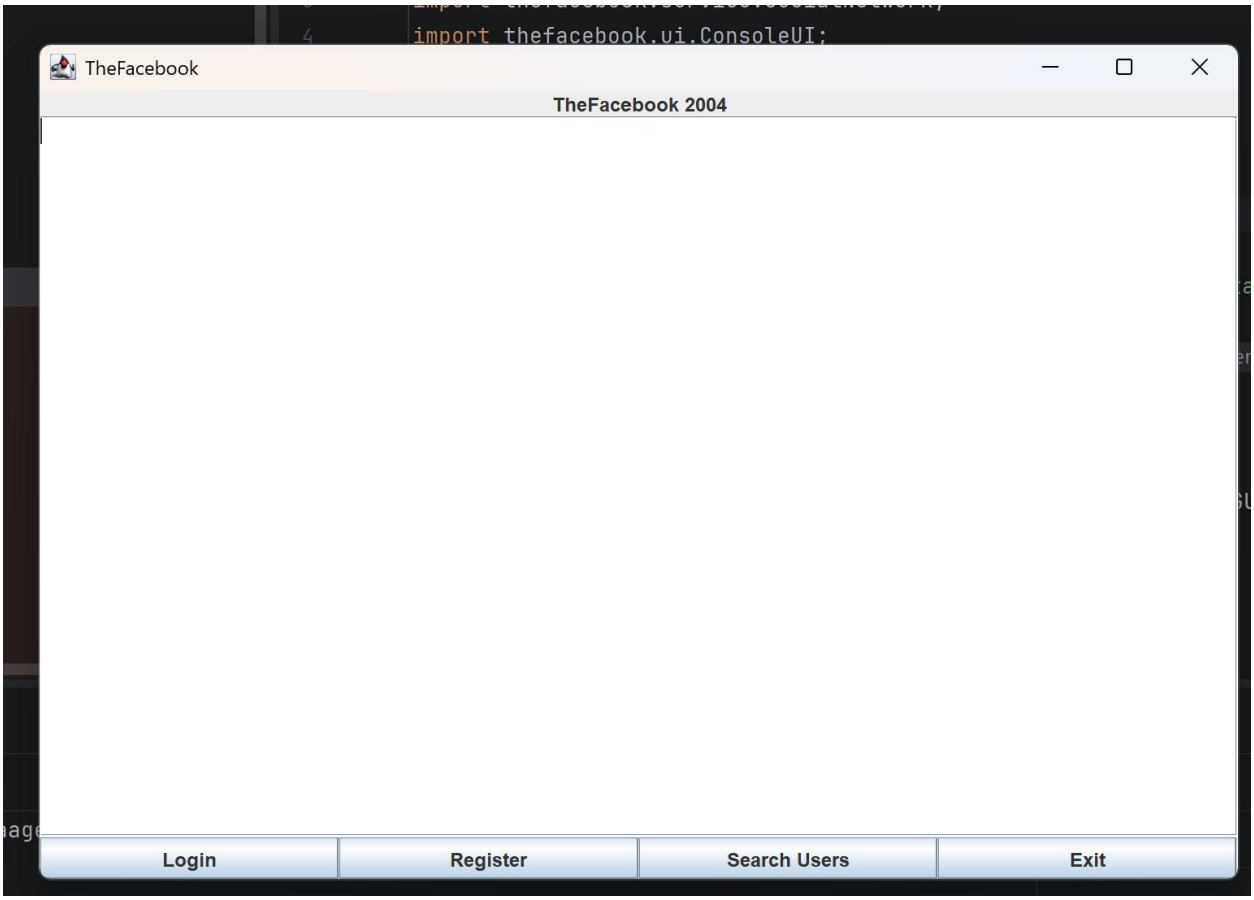
Problem	Solution
Direct use of HashMap was not allowed by the assignment.	A custom generic MyHashTable class was implemented using buckets and linked entries. It supports put, get, remove, keys, values, clear, and resize.
Friend relationships are difficult to manage using only arrays or lists.	A graph was used. Each user is a node and each friendship is an undirected edge. This makes mutual friends and connection degree easier to implement.
Data should not disappear after the program closes.	CsvStorage was created to save users, friendships, requests, private messages, group records, and group messages into CSV files, then load them again when the program starts.
Passwords should not be stored as plaintext.	PasswordUtil hashes passwords with SHA-256 before saving them to users.csv.
The console interface was not attractive enough as a social media application.	A simple Swing GUI was added. It uses basic components so it is easy to understand and suitable for a first-year project.
The GUI and console could have duplicated the same logic.	Both interfaces call the same SocialNetwork methods where possible, so the business logic remains in one place.
Friend recommendations and degree display need to consider indirect connections.	Breadth-first search is used in FriendGraph to explore up to third-degree connections.
Chat and group data need ordered dynamic storage.	ArrayList is used for group members and messages because records can grow dynamically and are displayed in order.
Browsing history needs back and forward	LinkedListBrowsingHistory uses Java LinkedList

movement.	with a current index to support visit, show history, back, forward, and clear.
The program needed enough test users.	The seed data and CSV files contain 30 normal users and 2 admins, satisfying the mock data requirement.
Avatar files should remain available after upload.	The selected image is copied to data/avatars, and users.csv stores the copied file path instead of only referencing the original location.
Search should support hobbies and avoid empty keyword mistakes.	The search function now checks hobbies using hasMatchingHobby and returns no results for empty keywords.

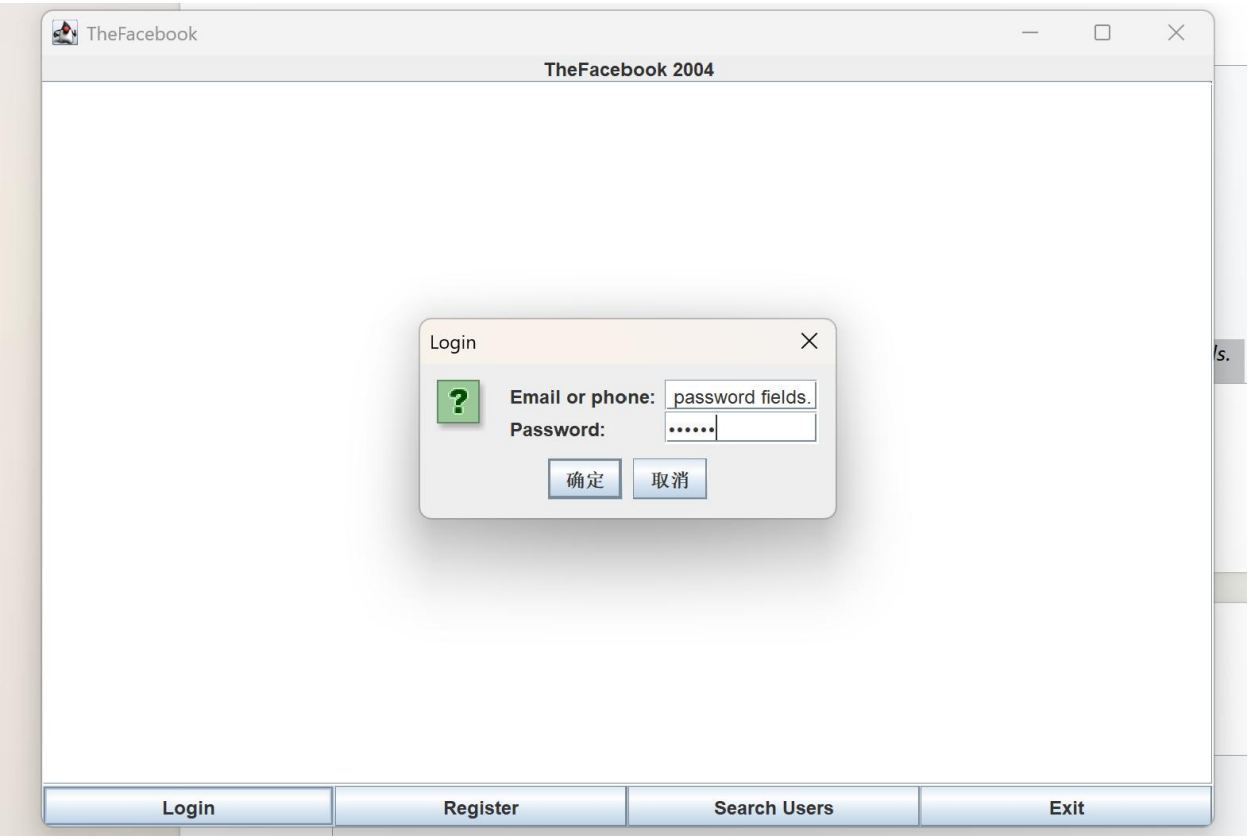
9. Program Screenshots

Please insert the actual screenshots into the blank boxes below before final submission. Each screenshot should show the full application window clearly.

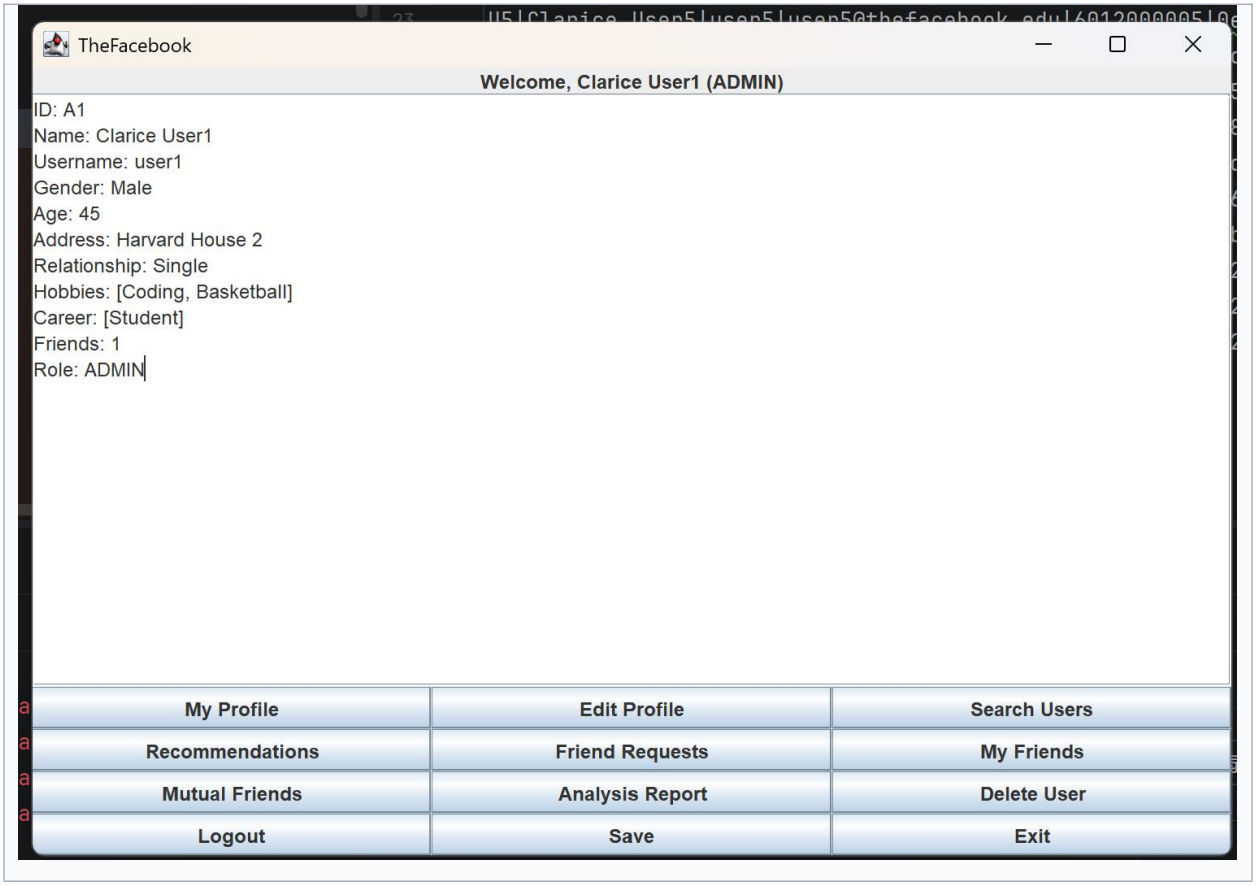
Screenshot 1: GUI Start Page



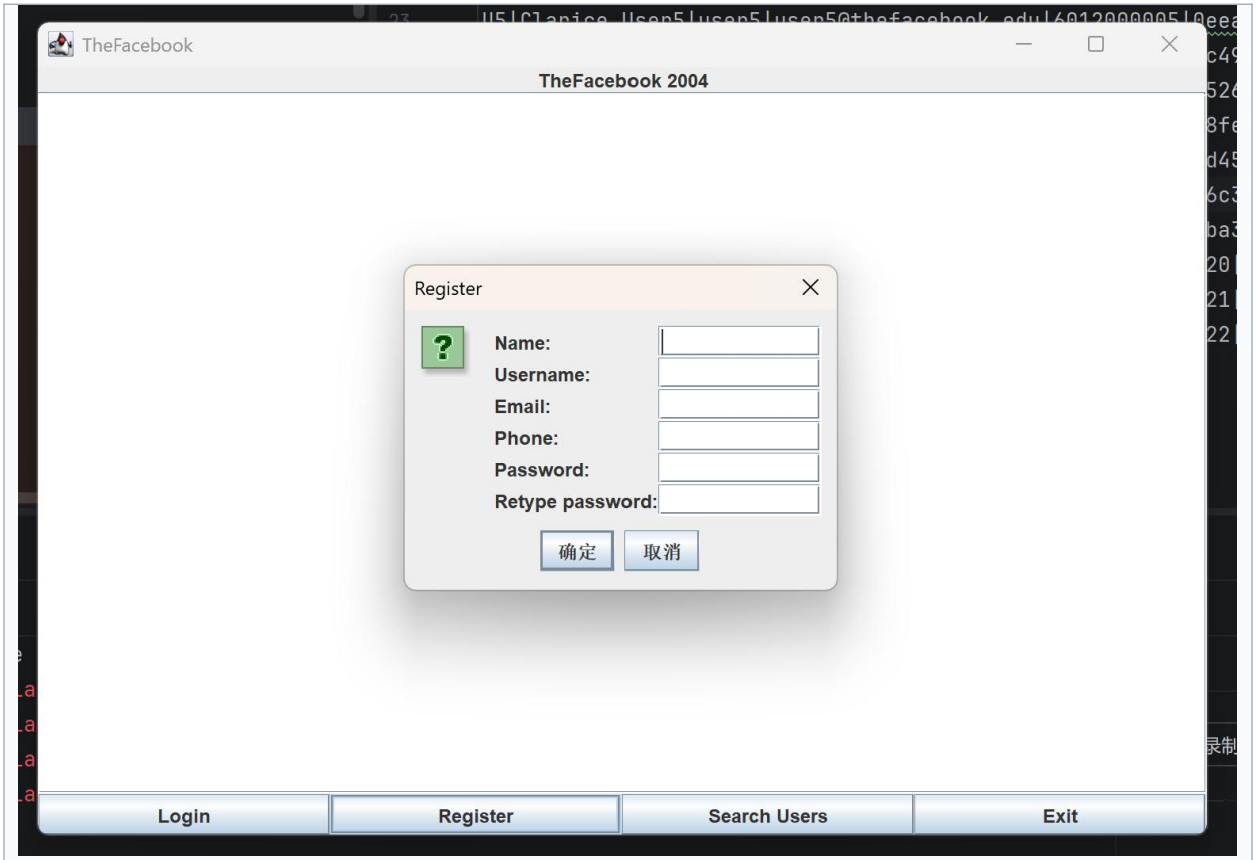
Screenshot 2: Login Dialog



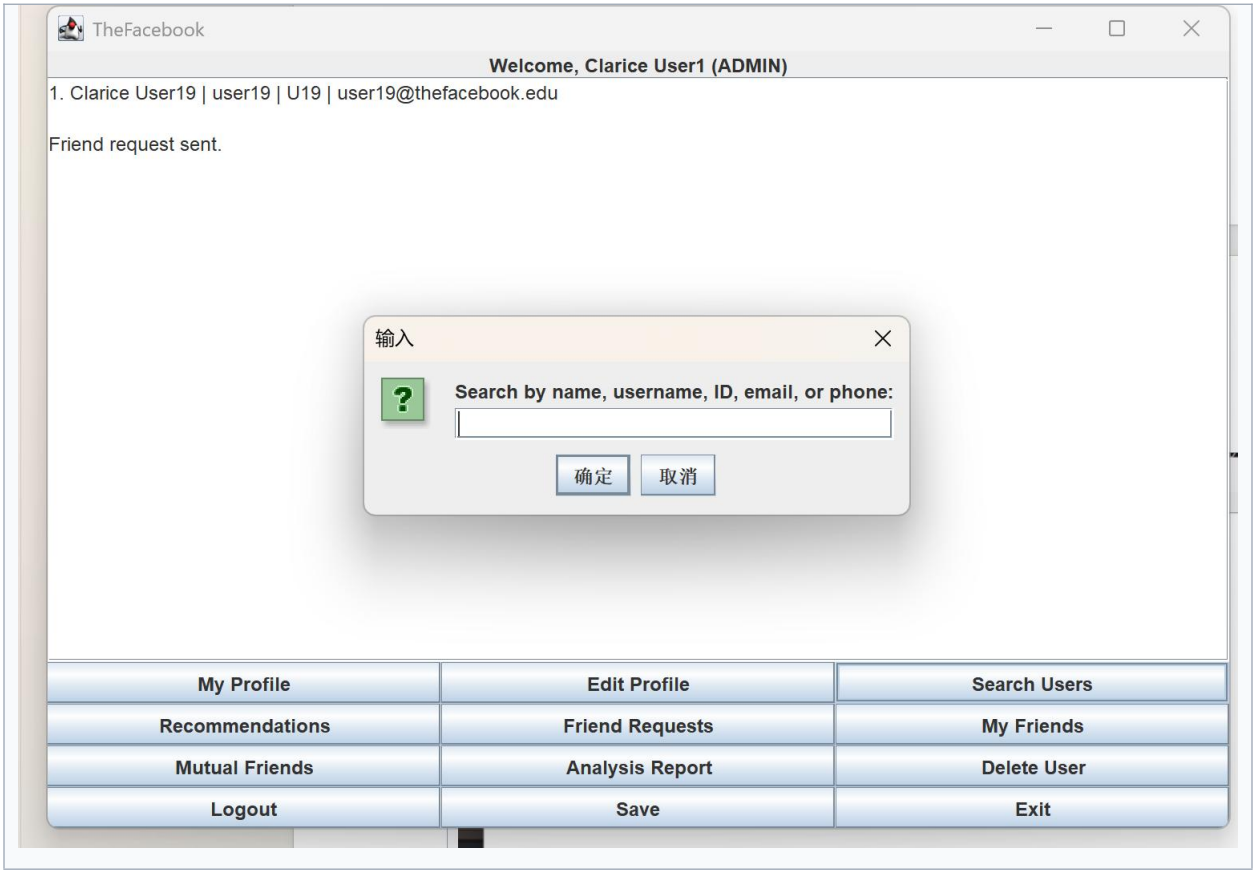
Screenshot 3: User Home Page



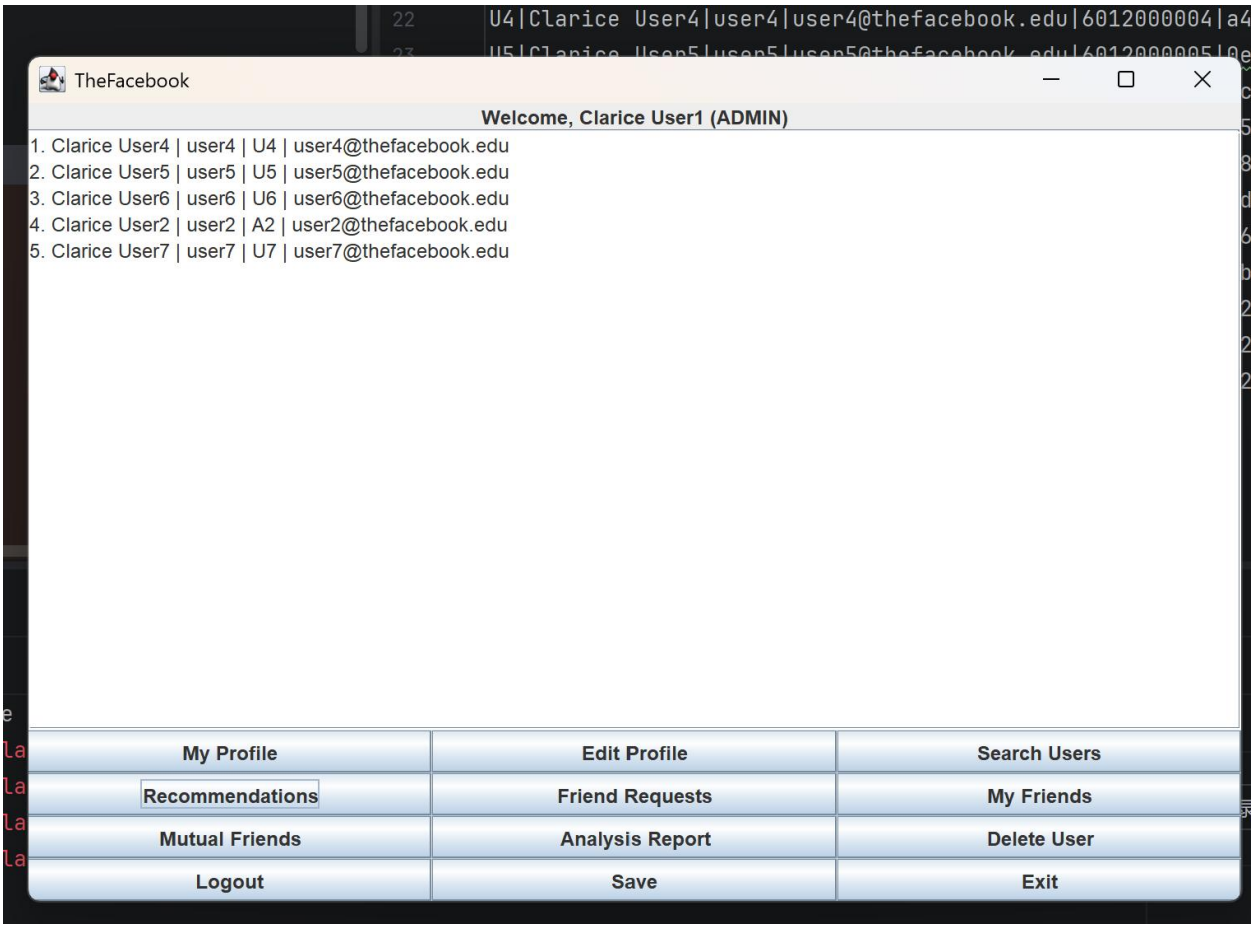
Screenshot 4: Register Function



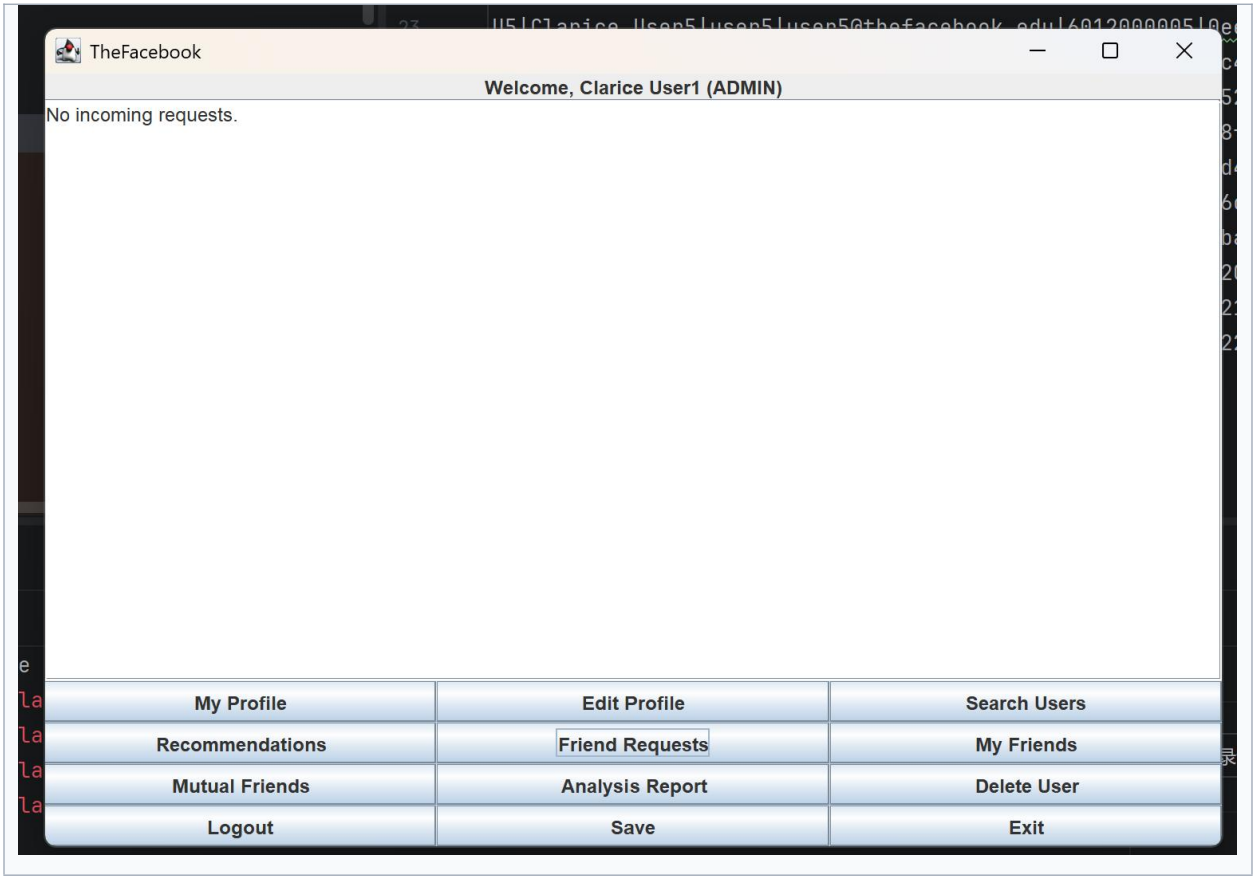
Screenshot 5: Search Users Function



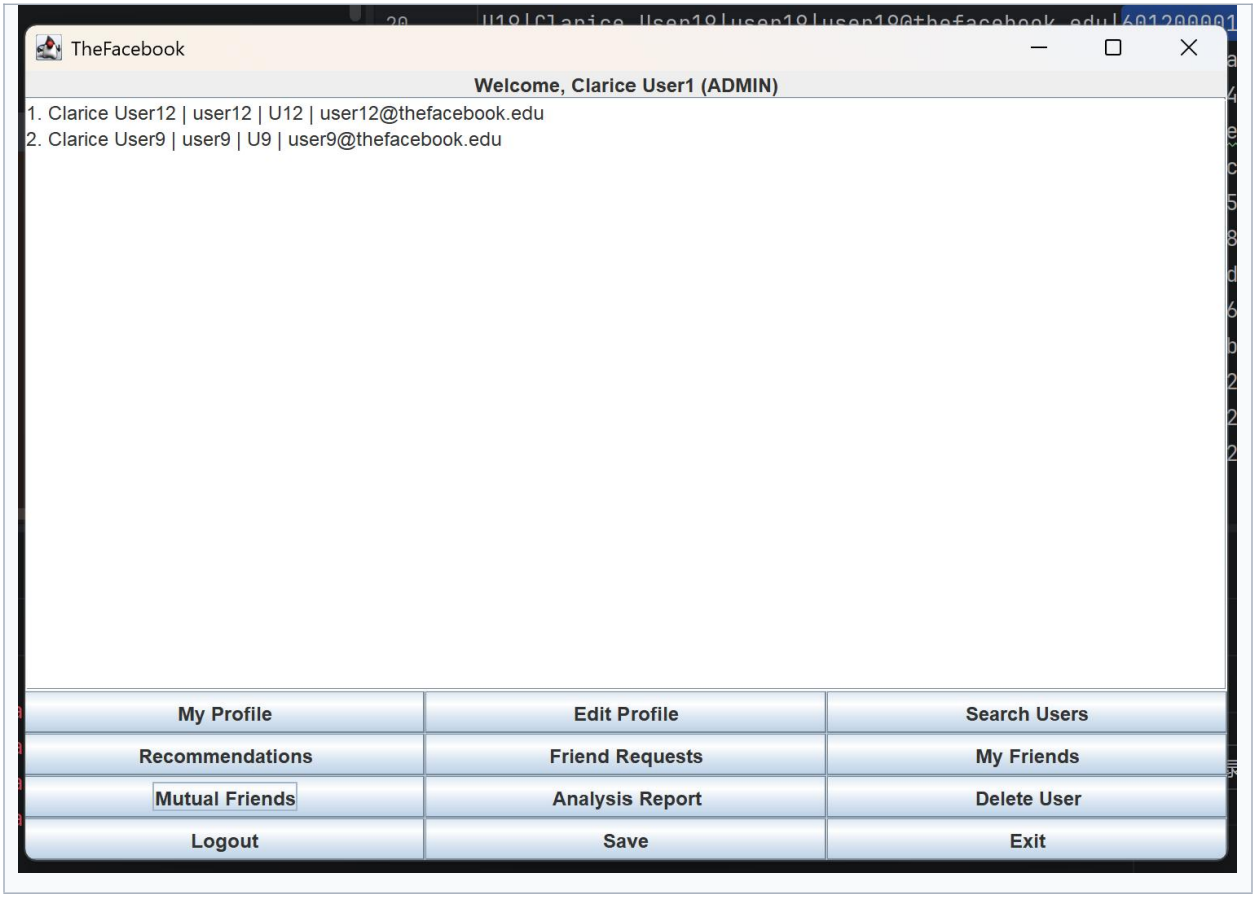
Screenshot 6: Friend Recommendations



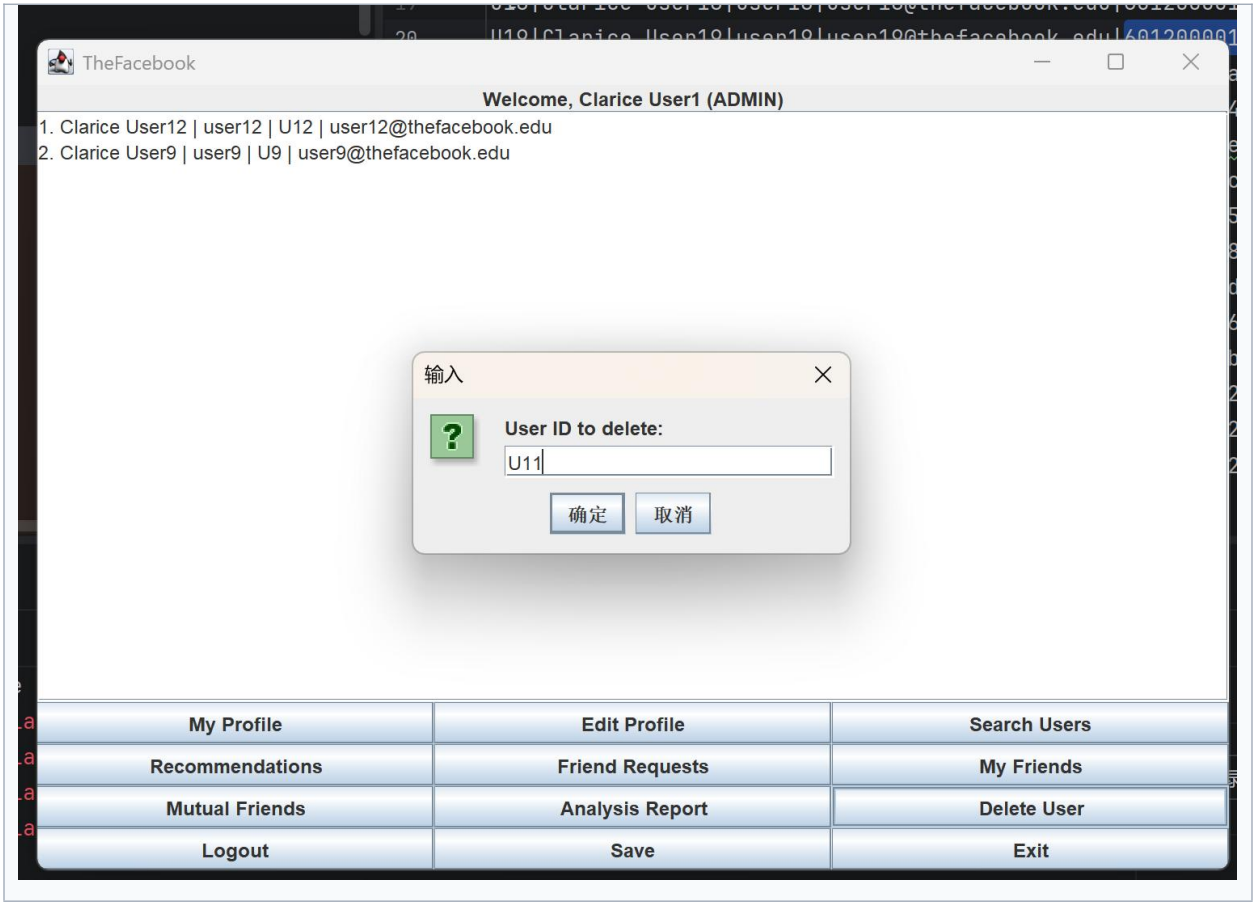
Screenshot 7: Friend Requests



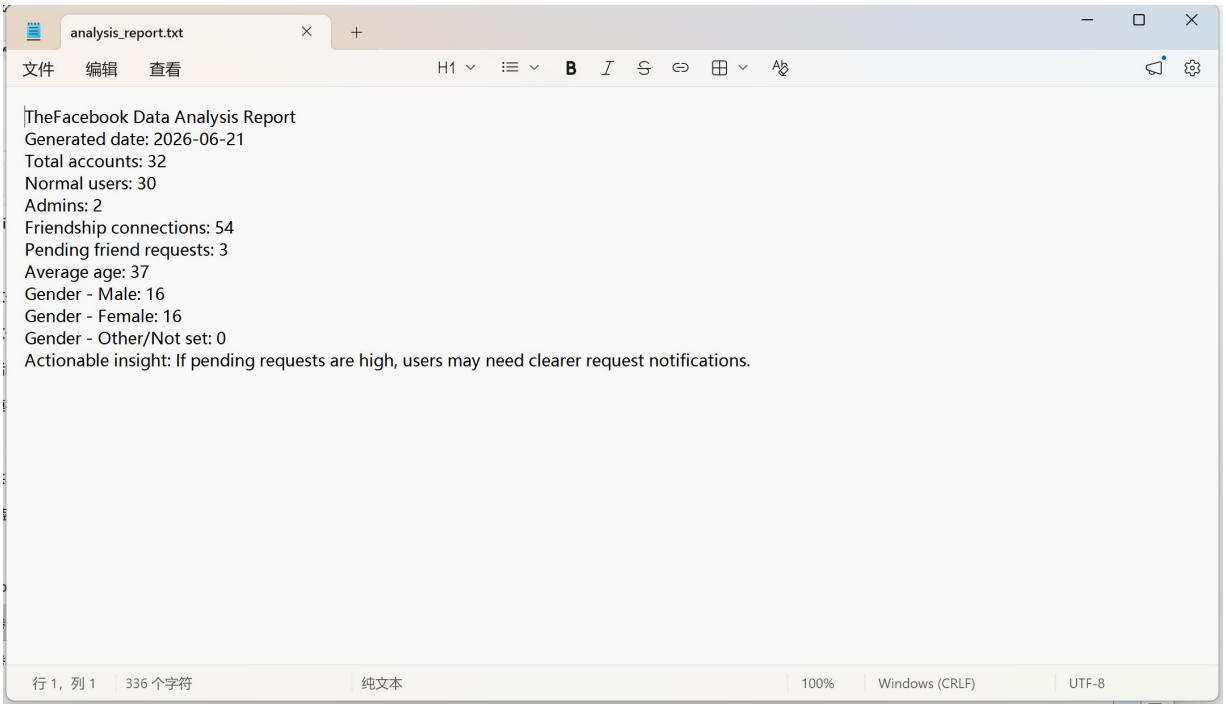
Screenshot 8: Mutual Friends



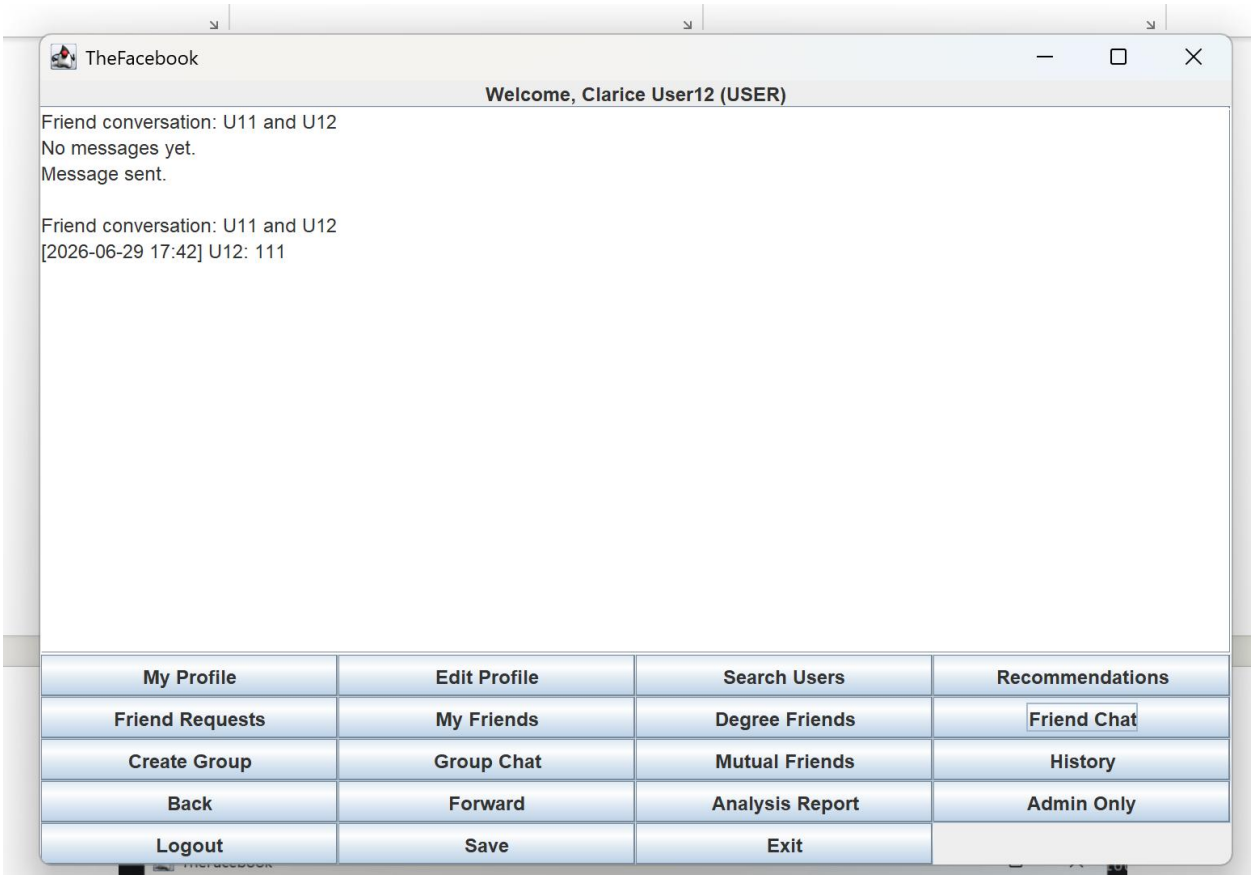
Screenshot 9: Admin Delete User



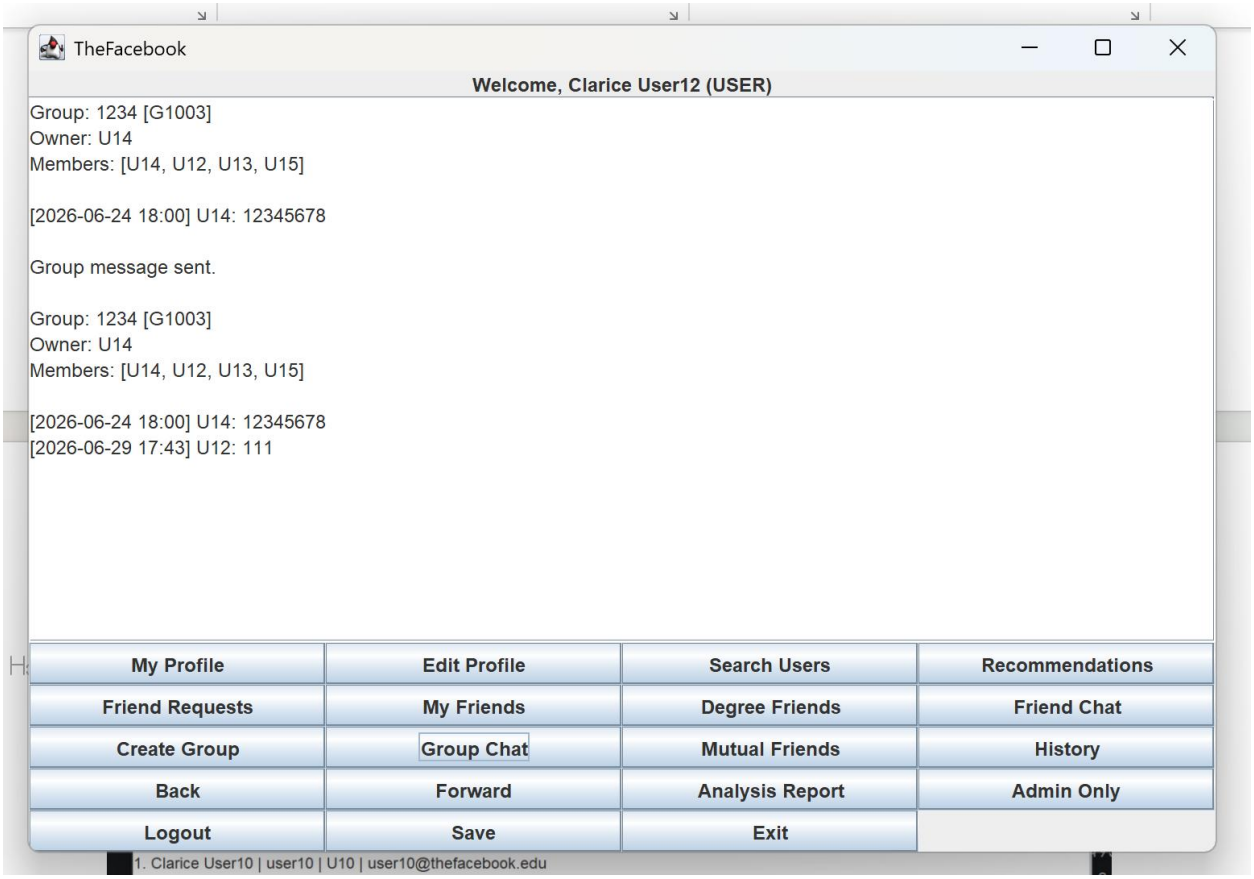
Screenshot 10: Data Analysis Report



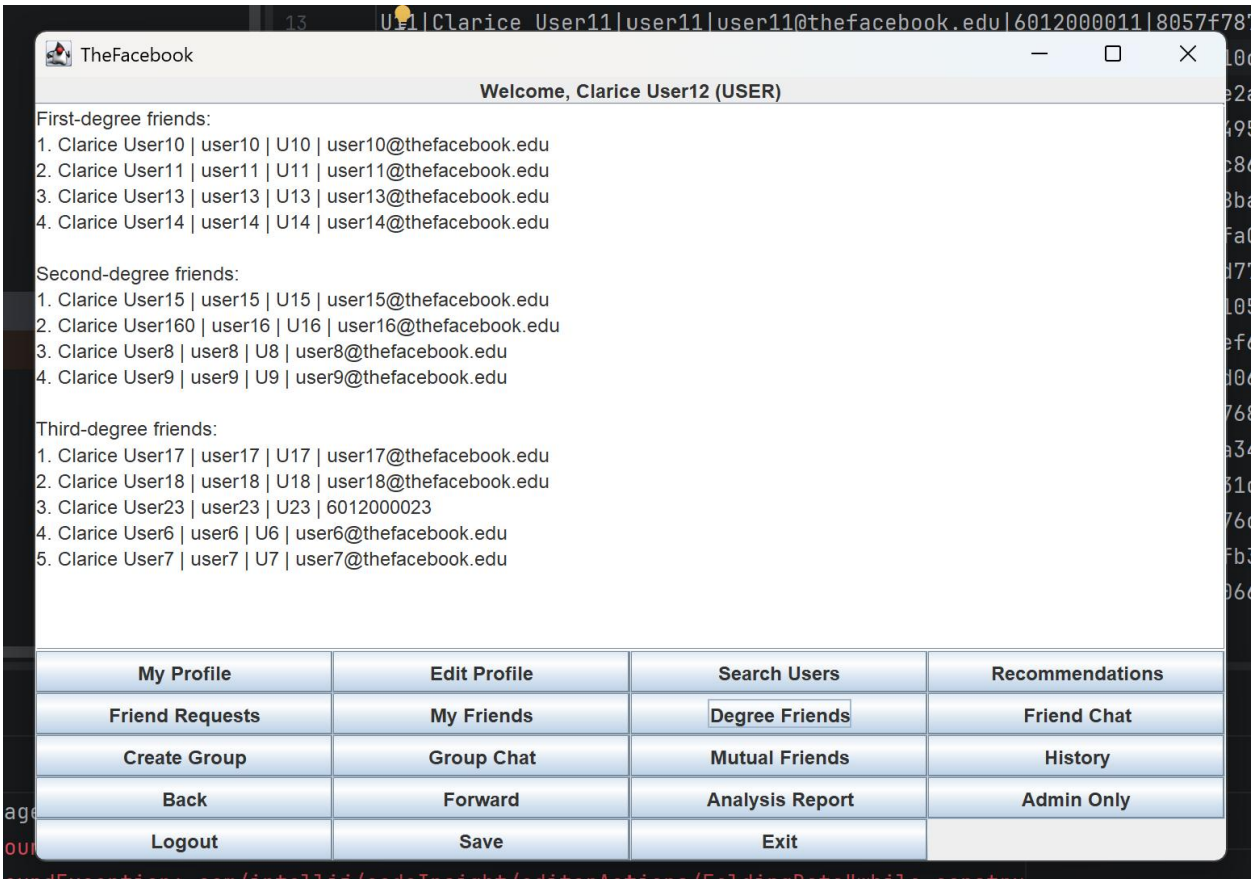
Screenshot 11: Friend Chat



Screenshot 12: Group Chat



Screenshot 13: Degree Friends



10. Testing Summary

- The Java source files were compiled successfully using javac after adding chat, group conversation, LinkedList browsing history, and degree friend display.
- The console version was tested by running the program with the console argument.
- The GUI version was tested by opening the Swing window and checking the main buttons and dialogs, including Friend Chat, Create Group, Group Chat, History, Back, Forward, Degree Friends, avatar upload, and hobby search.
- The data folder was checked and contains 32 mock users: 30 normal users and 2 admin users.
- The CSV files in the data folder were checked, including users.csv, friendships.csv, requests.csv, friend_messages.csv, groups.csv, and group_messages.csv.

11. Limitations and Future Improvements

- The GUI is intentionally simple. It can be improved with tables, better layout, profile pictures, and a richer chat window.

- The browsing history function stores text-area content snapshots and supports back/forward restoration. A future improvement would be to store page type and parameters so it can rebuild full Swing components for each previous page.
- Editing username, email, or phone number can be improved with duplicate checking.
- Future features can include more advanced fuzzy search, privacy settings, image messages inside chats, group member management, and a real database such as MySQL.

12. Conclusion

TheFacebook demonstrates how data structures can support a social networking application. The project uses a custom hash table for fast user and group lookup, a graph for friendship relationships and degree-based connections, a queue for friend requests, an ArrayList for dynamic collections, hobbies, and chat messages, a stack for career history, and a LinkedList for browsing history records. The Swing GUI improves usability and now supports avatar upload, hobby-based search, friend chat, group chat, browsing history restoration, and degree friend display. Overall, the program covers the major basic requirements of Topic 2 and includes meaningful extra features that can be explained clearly during presentation.